



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 09 - Part 1

Inheritance





THIS WEEK'S AGENDA

- 01. Inheritance
- 02. Derived Classes
- 03. Access Specifiers
- 04. Derived Class Example
- 05. Function Overriding





INHERITANCE





INTRODUCTION TO INHERITANCE

- In object-oriented design, **inheritance** is a relationship between a more general class (called the **base class**) and a more specialized class (called the **derived class**).
- The **base class** is sometimes also called the **superclass** or **parent class**, while the **derived class** is also called the **subclass** or **child class**.
- The derived class inherits data and behavior from the base class.
- Inheritance can be looked at as an “is-a” relationship between a general base class and a specialized derived class.



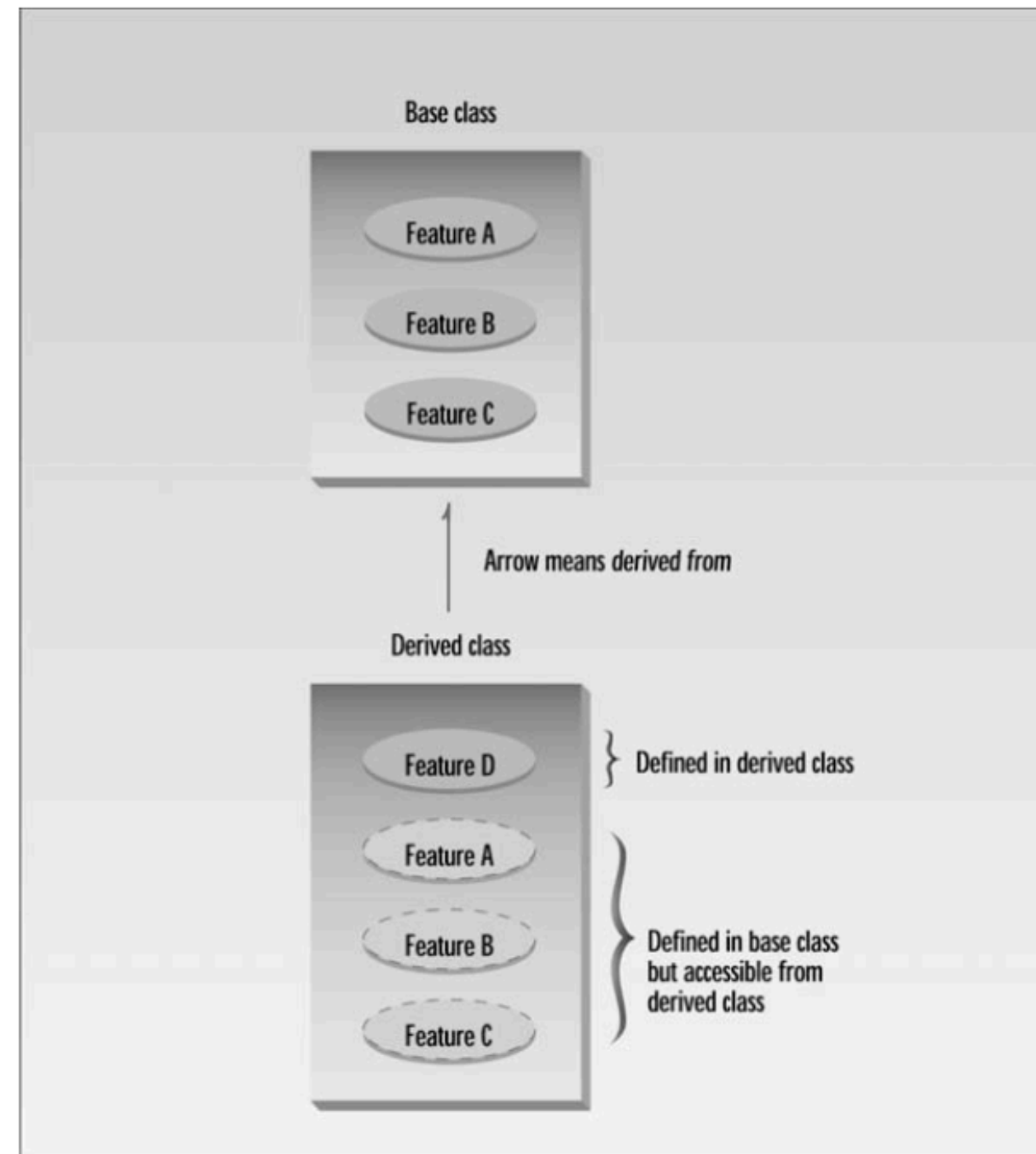
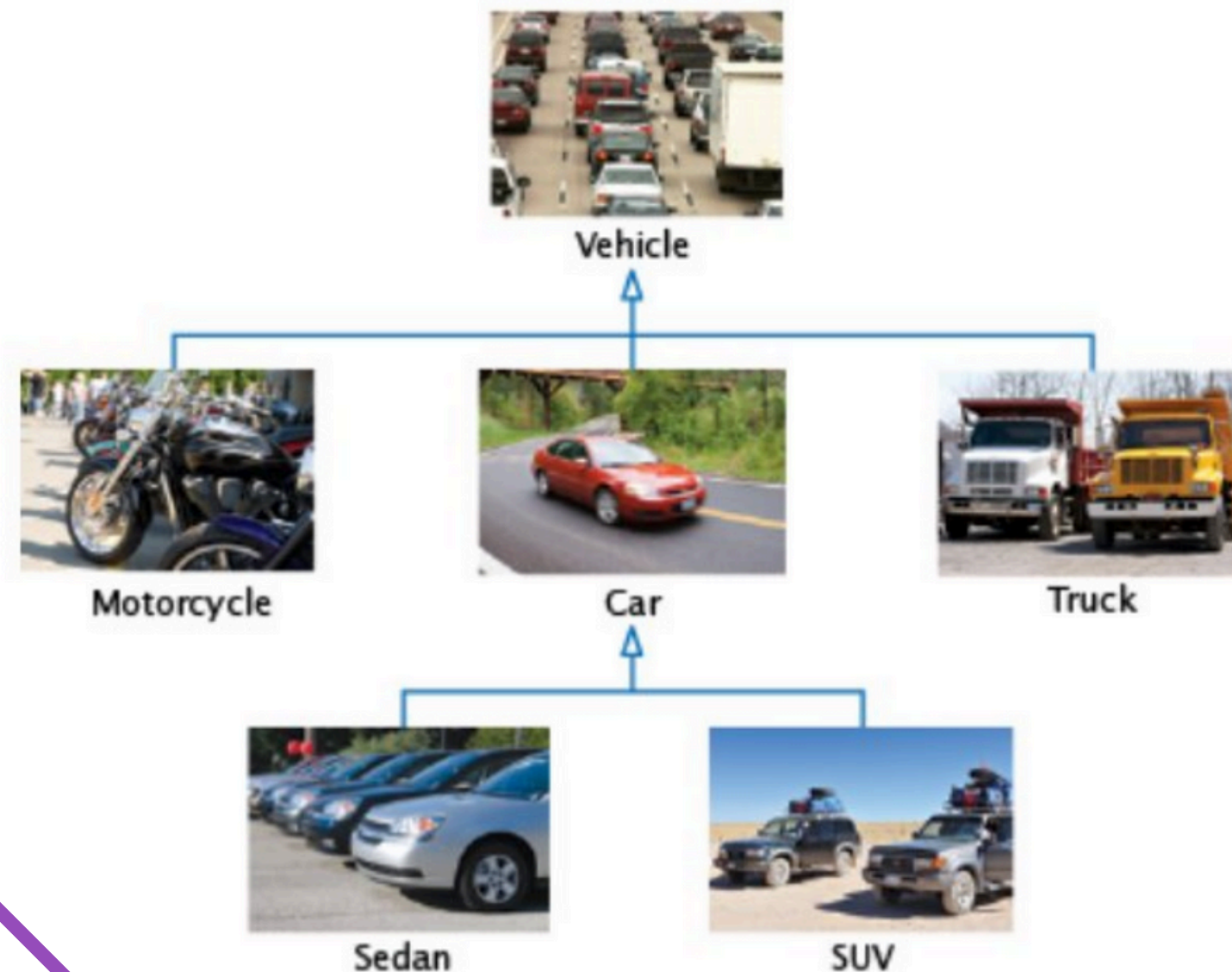


FIGURE 9.1
Inheritance.





THE INHERITANCE HIERARCHY



- Cars share common traits of vehicles, such as the ability to transport.
- The class Car inherits from the class Vehicle.
- The Vehicle class is the base class in this case and the Car class is the derived class.





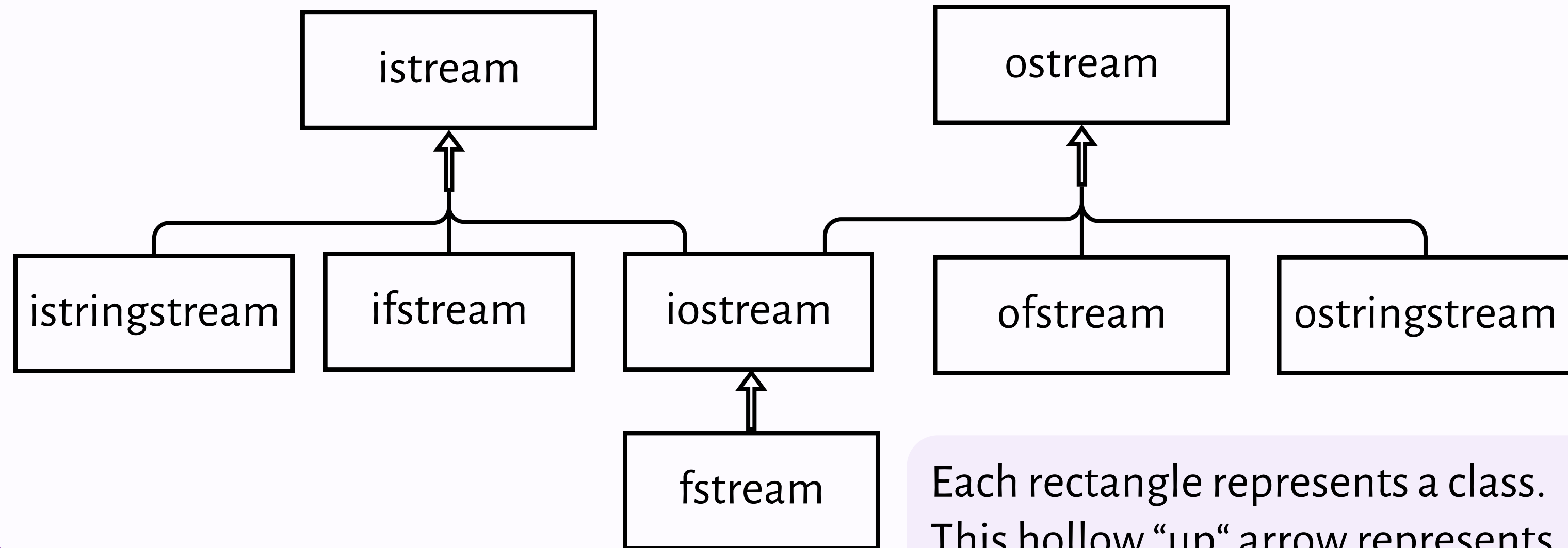
INTRODUCTION TO INHERITANCE

- The inheritance relationship is very powerful because it allows us to reuse algorithms with objects of different classes.
- Suppose we have an algorithm that manipulates a Vehicle object.
- Because a *car* is a special *vehicle*, we can supply a Car object to such an algorithm, and it will work correctly.
- This is an example of the **substitution principle** that states that you can always use a derived-class object when a base-class object is expected.





INHERITANCE HIERARCHY OF C++ IO CLASSES



Each rectangle represents a class. This hollow “up” arrow represents the inheritance relationship.

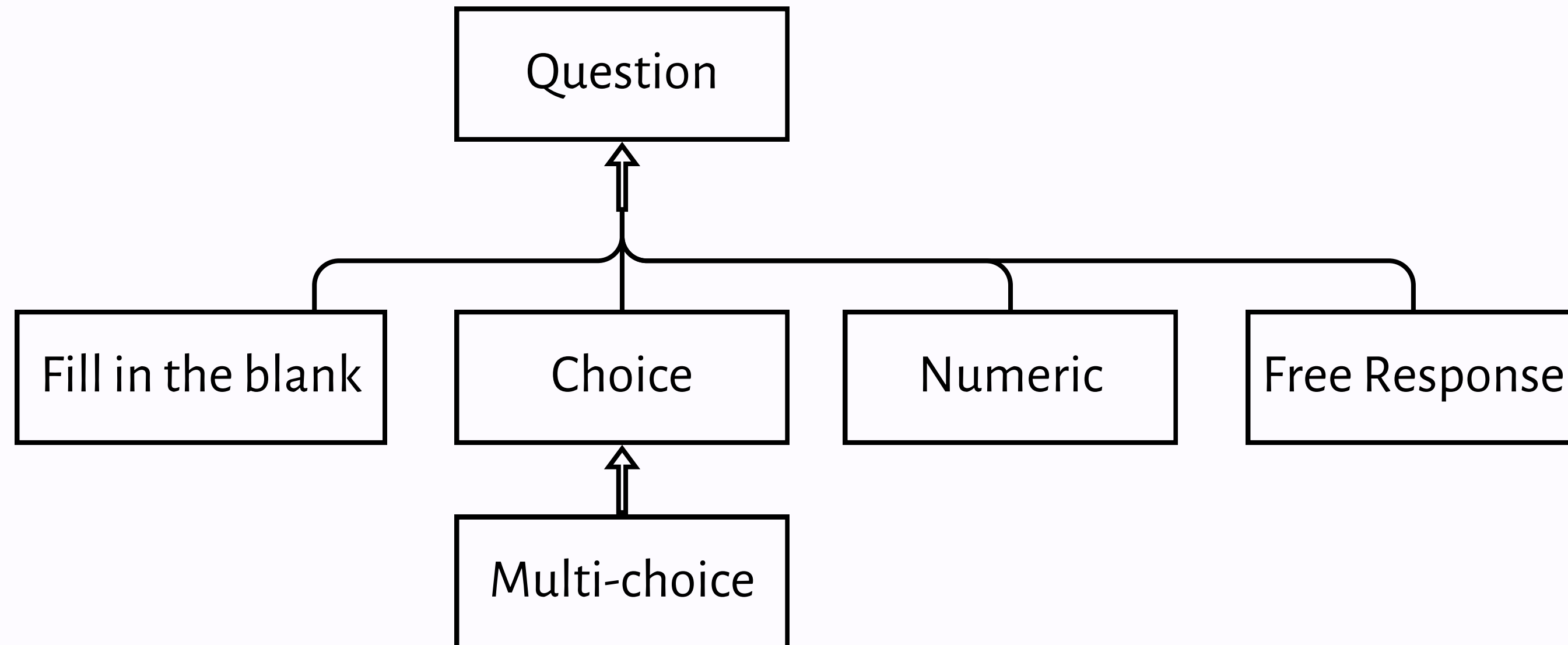


QUESTION HIERARCHY - EXAMPLE

- Consider a simple hierarchy of questions in a quiz. A quiz consists of different kinds of questions such as:
 - Fill in the blanks
 - Choice (single or multiple)
 - Numeric
 - Free responses
- An inheritance hierarchy for these question types can be constructed.



QUESTION HIERARCHY - EXAMPLE





DERIVED CLASSES





DERIVED CLASSES

- In C++, we form the derived class from the base class by specifying what makes the derived class different.
- We also define the member functions that are new to the derived class.
- The derived class inherits all member functions from the base class.
- However, we can change the implementation of the inherited member functions as per our need.
- The derived class automatically inherits all data members from the base class.
- We only have to define the additional data members.





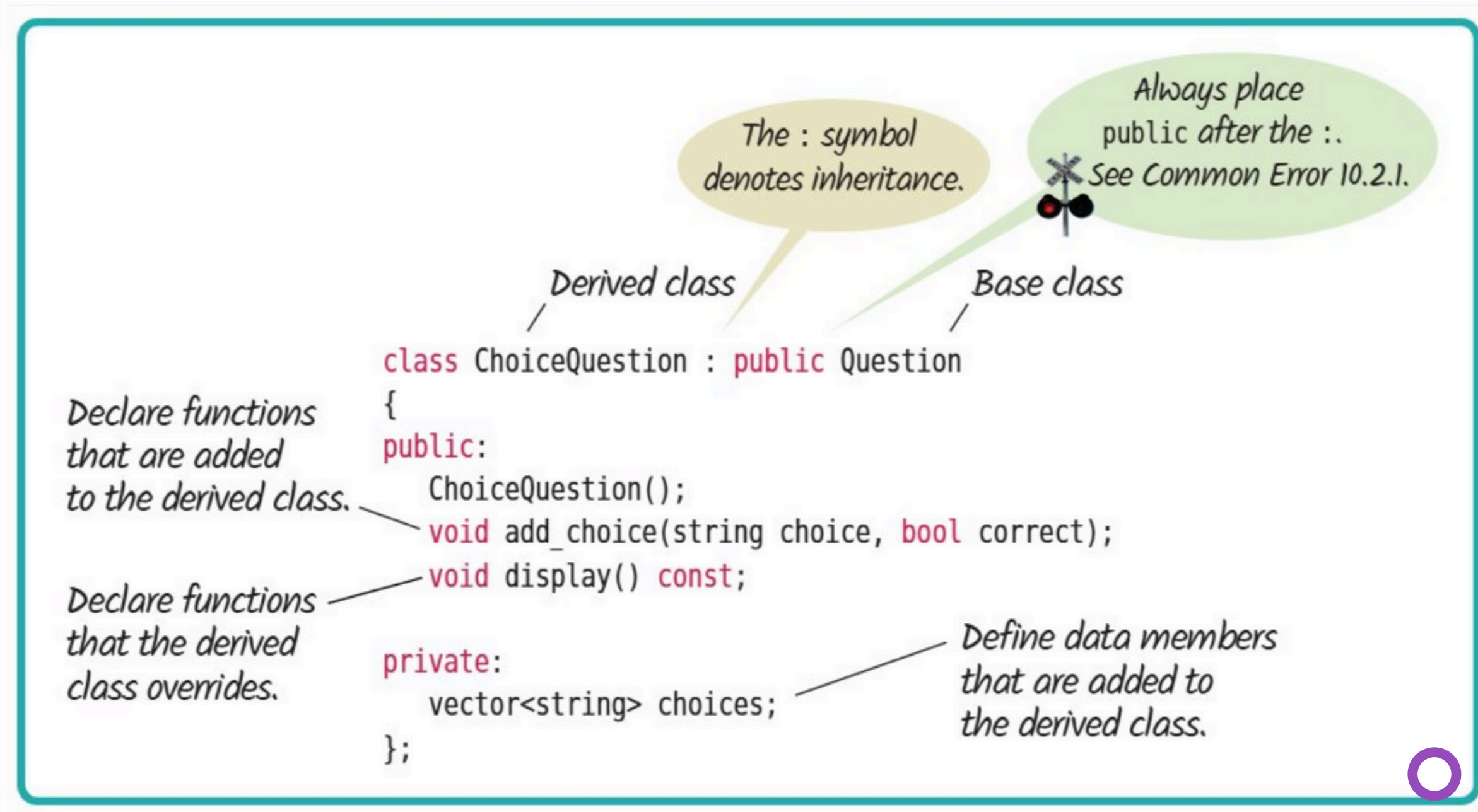
DERIVED CLASSES – EXAMPLE

- The syntax of a derived class is as follows:

```
class Child : public Parent {  
    public:  
        New and changed member functions  
    private:  
        Additional data members  
};
```



DERIVED CLASSES – FORMAT





ACCESS SPECIFIERS





ACCESS SPECIFIERS

- So far, we have been looking at two access specifiers: **public** and **private**.
- However, there is a third access specifier as well - **protected**.
- Let's look at each in detail:
 - **public** – class members declared as public can be accessed anybody.
 - **private** – class members declared as private can only be accessed by member functions of the same class or friends. ***This means that derived classes can not access private members of the base class directly.***
 - **protected** – class members declared as protected can be accessed by the class itself, friends, and also derived classes. They are not accessible from outside the class.





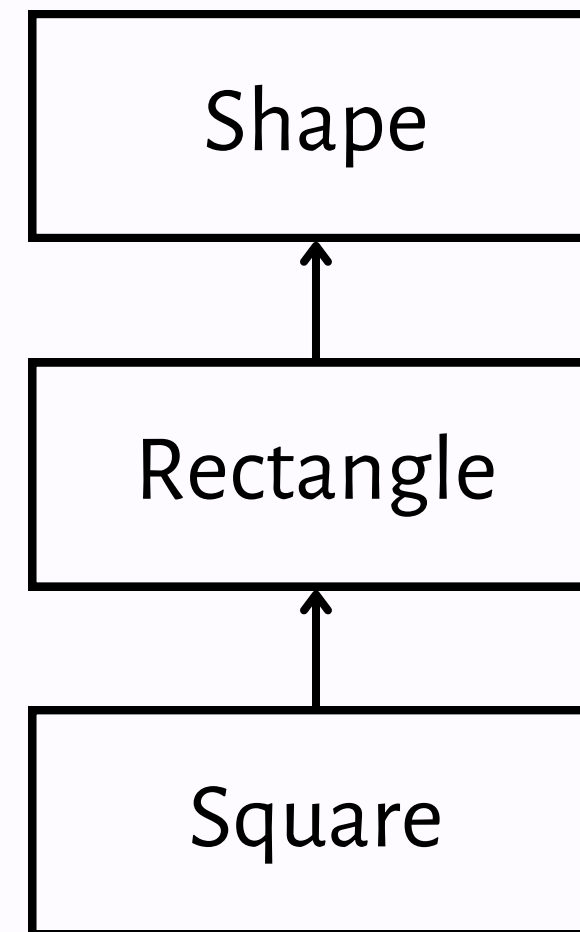
DERIVED CLASS EXAMPLE





DERIVED CLASS – EXAMPLE

- Consider the following inheritance hierarchy:





DERIVED CLASS – EXAMPLE

- Let's take a look at it in terms of how these classes are implemented in code.





DERIVED CLASS – EXAMPLE

- A Rectangle differs from a Shape in the following three ways:
 - It's objects store length and width, whereas objects of the Shape class do not.
 - It has a calculateArea member function for calculating the area of the rectangle.
 - The display function of the Rectangle class shows the area of the rectangle.





DERIVED CLASS – EXAMPLE

- The calculateArea function is specific to the Rectangle class.
- You can call it via the Rectangle objects, not the general Shape objects.
- However, the display function is a redefinition of a function that exists in the base class; it is redefined to take into account the special needs of the derived class.
- We say that the derived class **overrides** this function.
- ***Overriding a function refers to redefining a function from a base class in a derived class.***





OBJECT CREATION ORDER

```
53  int main() {  
54      Square sq;  
55      return 0;  
56  }
```

```
Constructor for Shape called..  
Constructor for Rectangle called..  
Constructor for Square called..
```





OBJECT CREATION ORDER

- The previous example shows that in order to create an object of a derived class, the constructor of its parent (base) class is called.
- This chain continues until we reach the base class.
- In effect, in order to create an object of type Square, we first create an object of type Rectangle, which creates an object of type Shape.
- ***What would happen if a constructor of a base class is not present?***





OBJECT CREATION ORDER

- What if we want to create a rectangle, and we want to assign it a name using its constructor. Can we do it directly as shown below?

```
Rectangle(string n, int l, int w) {  
    this-->name = n;  
    this-->length = l;  
    this-->width = w;  
    cout << "Creating a Rectangle.." << endl;  
}
```





OBJECT CREATION ORDER

- The correct way to initialize a parent(super/base) class's variables is to call its constructor, since the variables of the parent(super/base) class are private.
- This can be done using the initializer list method as follows:

```
Shape(string n) : name(n) {  
    cout << "Now creating a shape with name: " << name  
    << endl;  
}
```





OBJECT CREATION ORDER

```
Shape(std::string n) : name(n) {  
    std::cout << "Constructor for Shape with name: " << name << " called.." << std::endl;  
}
```

```
Rectangle(std::string n, int l, int w) : Shape(n), length(l), width(w) {  
    std::cout << "Constructor for Rectangle with name: " << n << " called.." << std::endl;  
}
```

```
Square(std::string n, int side) : Rectangle(n, side, side) {  
    std::cout << "Constructor for Square with name: " << n << " called.." << std::endl;  
}
```





OBJECT CREATION ORDER – RULES

- In C++, you cannot directly call the constructor of the superclass (base class) from within the body of the constructor of the subclass (derived class).
- Instead, you must call the base class constructor in the initializer list of the derived class constructor.
- There is a specific order to follow when initializing members in the initializer list of a constructor in C++.





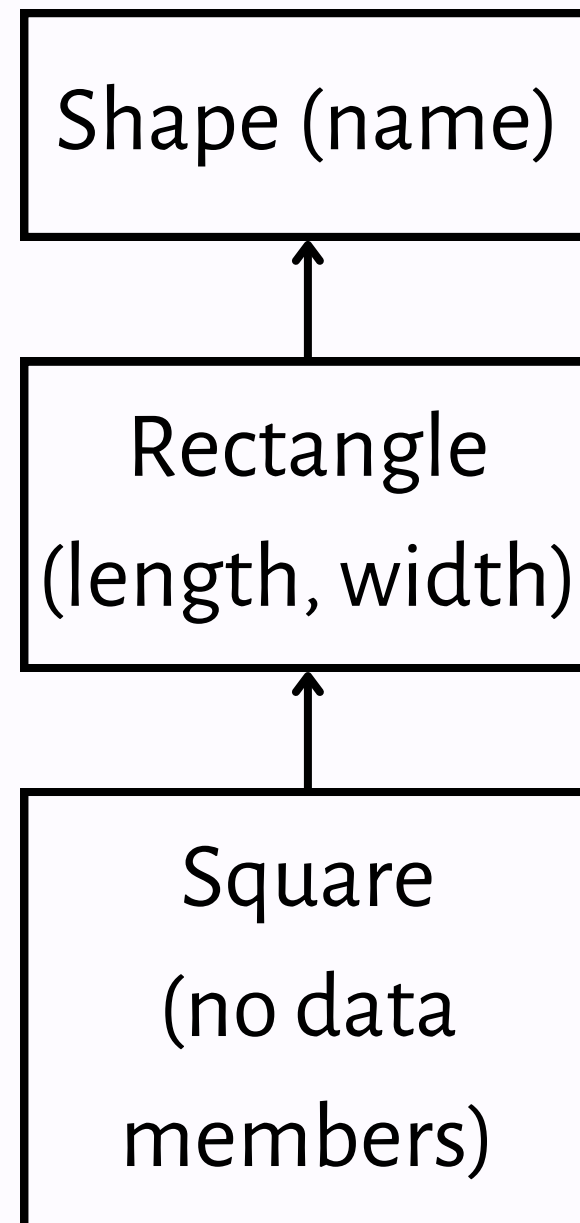
OBJECT CREATION ORDER – RULES

- The base class constructors are called before the derived class's member initializers are executed.
- If the derived class has multiple base classes, the order of initialization will follow the order in which the base classes are declared in the class definition (from left to right).





OBJECT CREATION ORDER – RULES



- In C++, when an object of a derived class is created, the constructors of its base class are invoked first.
- This ensures that the base class part of the derived class is fully initialized before any additional initialization is executed.
- The memory layout of a derived class object includes the memory for its base class parts as well.





OBJECT DESTRUCTION ORDER

- The order of object destruction follows the **reverse** of the construction order.
- When an object is destroyed, the destructors are called in the following order:
 - First the destructor of the derived class is called. This allows the derived class to perform any cleanup specific to itself.
 - After that, the destructor of the base class is called.
 - This ensures that any resources or states managed by the base class are properly cleaned up.





SUMMARY





SUMMARY

- Introduction to inheritance included the definitions of a base class and derived class, and the syntax of a derived class in C++.
- We also saw the order of object creation in the case of a base class and derived class.
- We also saw how functions of the base can be overridden in the base class.
- The order of object destruction is in reverse order of object creation.
- We also looked at the rules of object creation and how constructors of the base class are called in the initializer list.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 09 and 11





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

